

MOUSIKÉ

Relatório de Evolução

Frontend MVP e integração com o ecossistema da aplicação

Período documentado: 24 a 30 de setembro de 2026

Marco: construção do frontend funcional, integração social, histórico, recomendações e publicação de músicas

Projeto	Mousiké
Base documental	Backend MVP - 22/09/2026
Commit-base	4e2a221
Commit de fechamento	f14a2b8

30 de setembro de 2026

1. Resumo executivo

Este relatório registra o segundo grande marco técnico da Mousiké. Após a conclusão do Backend MVP, o foco do desenvolvimento passou para a construção de um frontend funcional em React e TypeScript, conectado às APIs existentes e capaz de executar os principais fluxos de uma plataforma de streaming musical.

No período documentado foram estruturadas a navegação pública e autenticada, o gerenciamento global de sessão, o layout persistente da aplicação, busca, reprodução de músicas, páginas de catálogo, biblioteca, favoritos, playlists, perfis, recursos sociais, histórico de reprodução, Home personalizada, recomendações e publicação de músicas.

Também foram realizados ajustes pontuais no backend para suportar a experiência do frontend, além de uma etapa de limpeza do repositório para remover resíduos do template Vite e corrigir a estrutura da Landing Page.

O resultado deste ciclo é um frontend de MVP amplamente funcional, ainda sem o design visual definitivo. A próxima etapa prevista é aplicar o sistema visual completo preparado no Figma - tipografia, logotipo, cores, componentes, telas, responsividade e Landing Page final - preservando a arquitetura e os fluxos já validados.

2. Panorama do marco

Área	Situação ao final do ciclo	Resultado principal
Frontend base	Concluída	React + TypeScript + Vite estruturados e roteados
Autenticação	Funcional	Sessão global, token, /auth/me e rotas protegidas
Catálogo e busca	Funcional	Busca integrada e páginas de música, artista e álbum
Player	Funcional	Player persistente com reprodução global e registro de histórico
Biblioteca	Funcional	Favoritos, playlists e operações principais
Social	Funcional	Seguir/deixar de seguir usuários e artistas
Recomendações	Funcional	Home personalizada consumindo o motor do backend
Publicação	Funcional	Upload de MP3/capa e criação automática do perfil Artist

Área	Situação ao final do ciclo	Resultado principal
Design final	Pendente	Aplicação do Figma na próxima etapa
Cadastro UI	Pendente	RegisterPage ainda é placeholder

3. Escopo e rastreabilidade

O relatório considera as alterações realizadas após a reorganização documental de 24/09/2026 e até o fechamento do frontend funcional em 30/09/2026. No intervalo, o branch principal avançou por cinco commits de produto/organização.

Commit	Data	Mensagem	Escopo
eeb59a8	24/09/2026	feat: inicia estrutura do frontend da Mousiké	Base React/TypeScript/Vite, React Router, Landing inicial e organização do frontend.
7e5f6d4	30/09/2026	feat: implementa autenticação e estrutura principal do frontend	Sessão global, rotas protegidas, layout interno, busca, catálogo e player persistente.
bdd9089	30/09/2026	feat: adiciona biblioteca playlists e recursos sociais	Favoritos, playlists, perfil, imagem de perfil e follows de usuários/artistas.
2ba540b	30/09/2026	feat: adiciona home personalizada histórico e publicação de músicas	Histórico ligado ao player, recomendações na Home e fluxo de publicação.
f14a2b8	30/09/2026	chore: limpa frontend e corrige estrutura da landing	Remoção de resíduos do Vite e correção da composição da Landing Page.

4. Base técnica do frontend

A camada de interface foi criada em `C:\Mousike\frontend`, separada do backend e preparada para consumir a API local durante o desenvolvimento.

- React 19.2.x como biblioteca de interface.
- TypeScript 6.x para tipagem estática no frontend.
- Vite 8.x como servidor de desenvolvimento e ferramenta de build.
- React Router 8.x para navegação e rotas protegidas.
- Oxlint para análise estática do código do frontend.
- Variável `VITE_API_URL` em `.env.local`; `.env.example` permanece versionável e sem segredos.
- Token JWT armazenado no `sessionStorage` sob a chave `mousike_access_token` para o MVP atual.

```
frontend/
├── src/
│   ├── components/
│   ├── contexts/
│   ├── hooks/
│   ├── layouts/
│   ├── pages/
│   ├── routes/
│   ├── services/
│   ├── types/
│   └── utils/
├── .env.example
├── package.json
└── vite.config.ts
```

5. Arquitetura da aplicação e navegação

O roteamento foi dividido entre a área pública e a área autenticada. O `App.tsx` delega a navegação para `AppRoutes`, enquanto `BrowserRouter` e os providers globais são montados no ponto de entrada da aplicação.

- `/` - Landing Page pública.
- `/login` e `/register` - autenticação e cadastro.
- `/app` - Home autenticada.
- `/app/search` - busca.
- `/app/history` - histórico de reprodução.
- `/app/library` - biblioteca, favoritos e playlists.
- `/app/profile` - perfil do usuário autenticado.
- `/app/publish` - publicação de músicas.
- `/app/songs/:songId`, `/artists/:artistId`, `/albums/:albumId`, `/playlists/:playlistId` e `/users/:userId` - páginas de detalhe.

A área autenticada utiliza um shell persistente composto por AppHeader, AppSidebar, Outlet e AppPlayer. LibraryProvider, SocialProvider e PlayerProvider ficam acima das páginas internas, permitindo que estado de biblioteca, relações sociais e reprodução sobrevivam à troca de rota.

6. Autenticação e sessão

- AuthProvider centraliza o usuário autenticado e o estado de carregamento da sessão.
- Login chama POST /auth/login, recebe token + usuário e grava a sessão.
- Ao atualizar a página, GET /auth/me reconstrói o usuário autenticado a partir do token existente.
- ProtectedRoute bloqueia /app quando não há usuário válido e redireciona para /login.
- Logout remove o token e retorna o usuário ao fluxo público.
- updateUser permite refletir alterações de perfil imediatamente no contexto global.

Decisão de MVP: sessionStorage foi mantido para simplificar o fluxo enquanto a aplicação permanece em desenvolvimento. Para produção, o mecanismo poderá ser reavaliado, por exemplo com cookies HttpOnly.

7. Integração com catálogo, busca e mídia

O frontend passou a trabalhar com serviços tipados para separar chamadas HTTP da camada de apresentação. O módulo api.ts centraliza a URL da API, leitura do token, construção de headers e normalização de URLs de mídia.

- Busca integrada à rota /search?q=... com resultados para músicas, artistas e álbuns.
- Páginas de detalhe reutilizam useResourceById para carregamento por ID, validação e tratamento de erros.
- TrackItem, SongCard, ArtistCard e AlbumCard concentram renderização e ações reutilizáveis.
- getMediaUrl converte caminhos relativos de /uploads/... em URLs completas do backend.
- Links entre entidades permitem navegar de música para artista/álbum e entre páginas relacionadas.

8. Player persistente

O PlayerProvider mantém a música atual em contexto global. Assim, a reprodução continua disponível enquanto o usuário navega entre páginas internas.

- PlayerSong passou a carregar id, título, duração, audioUrl, coverUrl e nomes dos artistas.
- AppPlayer resolve as URLs de áudio/capa e renderiza o elemento <audio> com controles nativos.
- playbackId diferencia reproduções distintas, inclusive quando a mesma música é tocada novamente.
- O player foi conectado ao histórico: ao atingir 25% da duração, registra uma reprodução no backend uma única vez por playback.

9. Biblioteca, favoritos e playlists

A biblioteca deixou de ser apenas estrutural e passou a executar operações reais contra o backend.

- Listagem e alternância de músicas favoritas.
- Criação de playlists com nome, descrição e visibilidade pública/privada.
- Listagem das playlists do usuário, incluindo privadas por meio de rota protegida específica.
- Adição de músicas a playlists diretamente de SongCard e outros pontos do catálogo.
- Remoção de músicas de playlists com atualização imediata da interface.
- Exclusão de playlists pelo proprietário.
- PlaylistPage respeita propriedade e privacidade ao disponibilizar ações administrativas.

10. Perfis e recursos sociais

- ProfilePage passou a editar username e bio.
- Upload e remoção de imagem de perfil com multipart/form-data.
- UserPage exibe perfil público de outros usuários.
- FollowUserButton implementa seguir/deixar de seguir usuários.
- ArtistPage passou a integrar FollowArtistButton.
- SocialProvider carrega e mantém artistas e usuários seguidos pelo usuário autenticado.
- O próprio usuário não recebe botão para seguir a si mesmo; o proprietário do Artist também não recebe botão de follow no próprio perfil.

A área de perfil também exibe indicadores simples da atividade atual: quantidade de favoritos, playlists, artistas seguidos e usuários seguidos.

11. Histórico de reprodução

O histórico, já disponível no backend, foi finalmente conectado ao comportamento real do player no frontend.

- POST /users/:id/history recebe songId e secondsListen.

- O backend registra a entrada apenas quando secondsListen alcança pelo menos 25% da duração total.
- AppPlayer calcula o limite localmente para evitar requisições desnecessárias antes do ponto mínimo.
- HistoryPage lista as reproduções do usuário em ordem recente, com música, data/hora e segundos ouvidos.

12. Home personalizada e recomendações

A rota /app passou a consumir GET /home e deixou de ser uma página placeholder. O backend agrega o motor de recomendações em seções prontas para apresentação.

- Mix para você.
- De artistas que você segue.
- Favoritas e parecidas.
- Porque você ouviu.
- Descobertas para você.

O motor usa sinais como histórico recente, favoritos, artistas seguidos, artistas e gêneros recorrentes e estratégias de cold start/fallback. Com isso, o produto passou a ter um ciclo de personalização: ouvir -> histórico -> sinais -> recomendações -> novas reproduções.

13. Publicação de músicas e criação de Artist

A página /app/publish foi transformada em um fluxo funcional de upload, conectado à rota POST /songs/publish.

- Na primeira publicação, o usuário informa o nome artístico; o backend cria o perfil Artist associado à conta.
- O arquivo de áudio é enviado como MP3 e a duração é obtida pelo backend, sem campo manual no frontend.
- A capa é opcional e aceita JPEG, PNG ou WEBP.
- Nas publicações seguintes, o frontend identifica o Artist do usuário pela rota protegida GET /artists/me.
- O artista do usuário é automaticamente enviado como role "main".
- Artistas existentes podem ser buscados e adicionados como colaboradores com role "featured".
- Após a publicação bem-sucedida, a interface navega diretamente para a página da nova música.

14. Ajustes realizados no backend

Embora o foco deste ciclo tenha sido o frontend, alguns ajustes no backend foram necessários para oferecer contratos adequados à interface.

- Adição de GET /users/:id/playlists para retornar todas as playlists pertencentes ao usuário autenticado, inclusive privadas.
- Adição de GET /artists/me antes de /artists/:id para identificar com segurança o perfil Artist da conta autenticada.
- Manutenção das rotas já existentes de favoritos, follows, histórico e recomendações como base da integração do frontend.
- Validação contínua do backend por npm run typecheck e npm run build.

Observação sobre ordem de rotas: /artists/me precisa ser registrado antes de /artists/:id, evitando que a string "me" seja interpretada como parâmetro numérico.

15. Qualidade, limpeza e organização do repositório

Ao final do ciclo foi realizada uma auditoria de higiene do repositório, especialmente no diretório frontend.

- Remoção do README padrão gerado pelo template Vite.
- Remoção de favicon.svg e icons.svg não utilizados.
- Remoção de hero.png e vite.svg que não eram referenciados pela aplicação.
- Correção de LandingPage.tsx, que continha uma cópia antiga do fluxo de login.
- LandingPage voltou a compor LandingHero + LandingFeatures dentro de LandingLayout.
- Confirmação de que node_modules, dist, .env, uploads, caches e artefatos gerados permanecem fora do Git.
- Package lockfiles e migrations permaneceram versionados por serem parte necessária da reprodutibilidade do projeto.

16. Validações executadas

Os principais blocos foram validados ao final do desenvolvimento com os comandos abaixo:

Frontend

npm run build

npm run lint

Backend

npm run typecheck

npm run build

Todos os comandos acima foram reportados como concluídos com sucesso antes do fechamento deste marco.

17. Ocorrências técnicas registradas

Ocorrência	Registro
PostgreSQL em recovery	Após interrupção de energia, o PostgreSQL 18 iniciou em estado de recovery e o login retornou erro 500 até o serviço ser normalizado. O problema foi identificado como operacional, não como defeito da aplicação.
Vite/Node com falta de memória Wasm	Em uma execução de npm run dev, o Node 24.21.0 retornou WebAssembly.instantiate(): Out of memory. A ocorrência foi tratada como problema de alocação de memória/processos/cache e não exigiu alteração da arquitetura do frontend.

18. Estado atual do produto

Ao final de 30/09/2026, a Mousiké possui uma base full-stack funcional para os principais fluxos de um serviço de streaming musical.

- Backend MVP completo e organizado.
- Frontend autenticado com navegação interna persistente.
- Busca e catálogo conectados à API.
- Player global funcional.
- Histórico alimentado pela reprodução real.
- Recomendações refletidas na Home.
- Favoritos e playlists operacionais.
- Perfis e follows operacionais.
- Publicação de músicas e criação de Artist operacionais.
- Landing estrutural corrigida, porém ainda sem design final.
- RegisterPage ainda não possui formulário funcional no frontend.

19. Pendências conhecidas e próximo marco

O próximo ciclo será predominantemente visual e de produto. O design completo será fornecido em Figma e passará a ser a fonte de verdade da interface.

- Implementar o design system vindo do Figma: tipografia, paleta, espaçamentos, componentes e tokens.
- Aplicar logotipo e identidade visual final da Mousiké.
- Reconstruir todas as telas atuais preservando seus fluxos funcionais.
- Finalizar a Landing Page conforme o layout definitivo.
- Implementar responsividade, estados de loading/erro/vazio e acessibilidade.
- Transformar RegisterPage em fluxo funcional de cadastro.
- Evoluir álbuns e gerenciamento de conteúdo publicado pelo artista.
- Executar polimento final, testes e preparação para deploy público.

20. Conclusão

O ciclo documentado representa a transição da Mousiké de um backend robusto para uma aplicação full-stack utilizável. O frontend deixou de ser uma estrutura inicial e passou a consumir praticamente todos os módulos centrais já construídos no servidor.

A arquitetura atual separa responsabilidades entre contextos, hooks, services, types, layouts e páginas, permitindo que a próxima etapa concentre esforço na experiência visual sem necessidade de reconstruir os principais fluxos de negócio.

Com o backend, a integração do frontend e a limpeza do repositório estabilizados, o projeto está preparado para receber a identidade final criada no Figma e avançar para a fase de acabamento, responsividade, acessibilidade e deploy.

Apêndice A - Estrutura funcional consolidada

Usuário

- > cadastro / login
- > sessão autenticada
- > Home personalizada
- > busca e catálogo
- > reprodução
- > histórico
- > favoritos e playlists
- > follows
- > recomendações
- > publicação de música
- > criação / uso do perfil Artist

Apêndice B - Convenções de validação

Frontend: npm run build + npm run lint. Backend: npm run typecheck + npm run build. O backend não possui script de lint configurado neste estágio; executar npm run lint nesse diretório resulta apenas em "Missing script: lint" e não representa falha do projeto.